Laboratório Funcional · Aula 12

Hugo Ernesto Contreras · RA 260151602 · Turma ESOFT6S-N-C
Gerado em 16/10/2026, 21:41:00 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1. "def agora(): return datetime.now()" É impura pois lê uma informação, a data atual, que está em constante mudança.
2. "def registra(msg): print(msg)" É impura pois altera um estado externo, fora do escopo da função, que no caso é o terminal. (ou pseudo terminal no caso do Linux, apenas por curiosidade).

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

sim

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

```
1: A-1 A-2
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Por conta da utilização da variável global, que sofre alteração pela função. Isso dificulta os testes por conta de não ser possível determinar exatamente o valor ao executar a função.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

f(x) = x + 2

g(x) = 3 * x

compor(f, g) (4)

(3 * x) + 2

(3 * 4) + 2

12 + 2 = 14

compor(g, f) (4)

3 * (x + 2)

3 * (4 + 2)

3 * 6 = 18

São gerados valores diferentes por conta da ordem dos fatores, (*, +) -> (+, *)

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

A função sorted(notas) segue o estilo funcional, pois não altera a lista original, que é um valor que existe fora da função, apendas devolve um valor com base no parâmetro.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

[B, C]

Previsão: (cons '(A B) '(C D))

[A, B, C, D]

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
- - - - - -
22
14
25
8
negativo
```

```
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) 79a5684cc80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 79a…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

Para identificar o tipo de dados como string.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(define (mostra x) (display x) (newline))
(mostra ( cdr (cdr '(A B C)) ))
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Eu acho que não vai funcionar em Scheme por alguma limitação de chamadas, mas em python vai executar normalmente.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

A pilha de execução na memória, na parte de HEAP.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme não faz o empilhamento na memória, o que não faz com que estoure a pilha de execução. No python, essa pilha chegou ao limite e causou erro.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond
    ((null? lis) '())
    ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
    (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

Entender a semântica da linguagem e a aplicação aninhada das funções que não conhecia.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

Acho que imprime os 5 primeiros e, depois 0.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
```

```
[0,1,4,9,16,25,36,49]
True
42

Time limit exceeded
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Bom, por dedução, acredito que a divisão possa ter retornado infinito e quem vem primeiro entre 42 e infinito é o 42.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

Porque o tempo limite de execução é alcançado, já que o valor 17 não é encontrado na lista o algoritmo tenta percorrer infinitamente, como a lista está ordenada, quando o elemento da lista se torna maior que 17, o algoritmo poderia ser encerrado com False.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

15, [5, 10, 15]

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Por conta do uso de funções anônimas que fica armazenada em uma variável.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Por conta da ordem de execução das operações matemáticas.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
                       .filter(n → n % 2 == 0)
                       .mapToInt(n → n * n)
                       .sum();

        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
// Prática 7 - Exercício 2: reescreva o laço com filter e map
// Cole em https://onecompiler.com/nodejs
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter(aluno ⇒ aluno.nota ⩾ 6)             // 1. Filtra quem tem nota ⩾ 6
  .map(aluno ⇒ aluno.nome.toUpperCase());      // 2. Extrai o nome e transforma em maiúsculas
```

```
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244
```

```
ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Após compreender a funcionalidade, a versão funcional me parece mais fácil por conta da sintaxe, já o laço é mais fácil de depurar por conta do acesso total a implementação.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em laço esta mais fácil de entender, por conta dos passos explícitos. Eu aceitaria a perda de performance em casos de trechos não críticos e que por requisito não funcional, não necessitam de micro otimizações.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A linguagem scheme que funciona com uma sintaxe muito peculiar.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro é de tipo, o compilador esperava uma lista e recebeu apenas um item, para corrigir deveria ser feito:
sort [b | b <- t, b <= h]